

C0328 Software Engineering Labs: Week 6 Worksheet

Course	C0328 Software Engineering
Week	06
Date	July 10, 2026
Duration	90 minutes (Work as a group)
Submission	Upload a PDF copy of your work by the end of the session, to the link provided on the course page
Objectives	By the end of this session, your group will: → Design test cases systematically, using equivalence partitioning and boundary value analysis, not just guesswork. → Write isolated, automated tests using whichever unit testing and mocking framework matches your stack. → Set up a CI workflow (GitHub Actions) that runs your tests automatically. → Critically review your own test suite as a group and identify what it does and doesn't catch. → Continue documenting your progress in the shared Google Doc and Trello board.

Tasks to be completed during the lab session

Step 1: Design Test Cases for Your Project

- As a group, review your Project's modules and use cases.
- Select 3-4 functions/methods that are critical and currently untested (e.g. business logic, calculations, validations, data transformations)
- For each selected function, apply proper test design techniques rather than picking inputs at random:
 - Equivalence partitioning : group inputs into classes that should behave the same way, and pick one representative value per class.
 - Boundary value analysis : test the edges of each valid range (just inside, just outside, and exactly on the boundary).
 - Error/negative cases : invalid types, nulls, empty inputs, and the exception or error each should trigger.
- Document the external dependencies each function touches (database, API call, file system, another class) that will need to be mocked or stubbed.
- Assign each group member a use case to model. Mention the name and index number.

Step 2: Implement and Run Isolated Automated Tests

- Each member writes automated unit tests for their assigned function, using their language's own testing framework.
- Tests must cover every case identified in Step 1 (equivalence classes, boundaries, and error cases) not just happy path.
- Run the test suite locally and record, for each function:
 - Number of tests written and number passing.
- As a group, merge everyone's tests into the shared repository and confirm the full suite runs together without conflicts.

Step 3: Automate with CI and Review as a Group

- Add a GitHub Actions workflow file (.github/workflows/tests.yml) to your project repository — the structure is the same whatever language you use.
- Configure the workflow to trigger on every push and pull request, install dependencies, run the full test suite, and fail if any test fails.
- Push a small change and confirm the workflow runs and reports a pass/fail status on GitHub.
- As a group, hold a short test review: swap functions with another member and try to think of one input that might still break the code but isn't covered by any existing test
- Here are some good online sources for further reading:
 - <https://site.mockito.org>
 - <https://jestjs.io/>

Instructions for Ongoing Documentation

- Continue updating the **shared Google Doc** under the new section titled: “**Week 6: Testing**”
 - For each tested function, indicate who worked on it (index number + name), the framework used, test counts/coverage, and any gap found in the Step 3 review.
 - Convert only this week's task (test code, run results, CI workflow screenshot, and review notes) into a PDF file and upload to the LMS.
 - Filename format: `CO328_GroupNumber_Week6.pdf`
 - Update your **Trello board** with relevant tasks (e.g., “Design Test Cases”, “Write Unit Tests”, “Set Up CI Workflow”, “Peer Test Review”).
-